I.E.S. CHAN DO MONTE
Marín, Pontevedra

PROYECTO DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA

MEMORIA DEL PROYECTO

ALUMNO
Álvaro Castiñeira Soto

TUTOR
Pablo Vázquez Fernández

CICLO
Desarrollo de Aplicaciones Multiplataforma

CURSO
2º curso

CURSO ACADÉMICO
2023–2026

2. ÍNDICE

3. INTRODUCCIÓN.... 4
4. OBJETIVOS Y ALCANCE.... 5
4.1. Objetivo general.... 5
4.2. Objetivos específicos.... 5
A) De producto (experiencia del deportista).... 5
B) De inteligencia artificial (IA aplicada al entrenamiento).... 5
C) De integraciones y datos.... 6
D) De arquitectura y calidad (Enfoque Híbrido).... 6
4.3. Alcance del proyecto.... 7
4.4. Suposiciones y dependencias.... 8
4.5. Riesgos y mitigación (resumen).... 8
4.6. Métricas de éxito (KPI).... 9
5. ESTUDIO EMPRESARIAL.... 9
5.1. La Idea: ¿Por qué nace OXI?.... 9
5.2. Identidad Visual y Marketing.... 10
5.3. Estudio de mercado y cliente objetivo.... 10
5.4. Análisis del entorno y de la competencia: DAFO y CAME.... 11
A) DAFO (Análisis interno/externo).... 12
B) CAME (líneas de acción).... 12
5.5. Ubicación.... 12
5.6. Forma jurídica y obligaciones legales.... 13
5.7. Recursos humanos y producción.... 15
5.7.1. Organigrama.... 16
5.7.2. Convenio colectivo.... 16
5.7.3. Prevención de riesgos laborales.... 17
5.7.4. Recursos materiales y humanos necesarios.... 17
5.8. Plan económico-financiero.... 18
5.8.1. Inversiones y gastos iniciales (aproximación).... 18
5.8.2. Gastos fijos y variables anuales.... 19
5.8.3. Fuentes de financiación, ayudas y subvenciones.... 19
5.8.4. Estimación de ingresos a un año.... 20
6. ESTUDIO TÉCNICO.... 21
6.1. Análisis y planificación.... 21
6.2. Requisitos hardware y software.... 22
6.3. Implementación.... 23
6.4. Funcionamiento general de la aplicación.... 24
6.5. Interfaces de la aplicación.... 24
6.6. Pruebas y validación.... 25

6.7. Implantación y despliegue....26
6.8. Medidas de seguridad y protección de datos....27
7. PROPUESTAS DE MEJORA.... 29
7.1. Propuestas de mejora técnicas....29
7.2. Propuestas de mejora de producto y negocio....30
8. DIFICULTADES ENCONTRADAS.... 31
8.1. Definición del alcance y riesgo de sobredimensionar el proyecto....31
8.2. Interoperabilidad Java-Kotlin y curva de aprendizaje....32
8.3. Diseño de la estructura de datos pensando en la IA....32
8.4. Gestión del tiempo y compatibilidad con otras obligaciones....33
9. CONCLUSIONES.... 33
10. BIBLIOGRAFÍA Y RECURSOS.... 34
10.1. Documentación técnica y desarrollo Android....34
10.2. Inteligencia artificial y análisis de datos.... 34
10.3. Negocio, mercado y emprendimiento....35
10.4. Normativa y protección de datos....35
11. ANEXOS.... 35
Anexo I: Prevención de Riesgos Laborales.... 35
Anexo II: Ficha Técnica del Entorno.... 36
Anexo III: Cronograma del Proyecto.... 37

3. INTRODUCCIÓN

En este proyecto se presenta OXI, una aplicación móvil multiplataforma orientada a deportistas de resistencia, especialmente corredores y triatletas. El objetivo de OXI es ayudar a planificar, organizar y seguir los entrenamientos de una forma sencilla, clara y coherente con la realidad del día a día: estudios, trabajo, vida personal, cansancio acumulado e imprevistos que obligan a ajustar el plan sobre la marcha.

La idea de OXI surge directamente de la experiencia directa como deportista de fondo. Durante años he gestionado mis entrenamientos combinando diferentes aplicaciones, hojas de cálculo, notas en el móvil y plataformas externas. Esta dispersión de la información hace que sea difícil tener una visión global de la carga de entrenamiento, analizar la evolución con el tiempo o reaccionar con rapidez cuando algo no sale como estaba previsto (exámenes, cambios de horario, molestias físicas, mala calidad del sueño, etc.). A partir de esta situación se detectó una necesidad clara: contar con una única herramienta que centralice la planificación, el registro y el análisis del entrenamiento de forma flexible.

OXI nace precisamente para cubrir esa necesidad. Por un lado, ofrecerá una vista estructurada de los entrenamientos a corto y medio plazo (semana y mes), permitiendo registrar no solo qué sesión se ha realizado, sino también cómo se ha realizado: sensaciones, nivel de fatiga, dolor, calidad del descanso o cumplimiento del objetivo previsto. Por otro lado, el proyecto está pensado desde el principio como una base sobre la que introducir progresivamente funcionalidades de inteligencia artificial. La idea es que, en fases posteriores, la aplicación pueda utilizar técnicas de IA para analizar los datos del usuario y proponer ajustes en la carga, detectar patrones de cansancio, avisar de posibles excesos de entrenamiento o sugerir modificaciones en los entrenamientos en función del historial y del contexto personal.

Además de responder a una necesidad real dentro del ámbito del deporte, OXI también tiene un objetivo formativo claro: servir como proyecto completo de desarrollo de aplicaciones multiplataforma. A través de esta aplicación voy a poner en práctica conceptos de arquitectura de software, persistencia de datos, diseño de

interfaces móviles, integración con servicios externos y, de forma especial, la preparación de la aplicación para trabajar con modelos de inteligencia artificial. De este modo, el proyecto se convierte no solo en una herramienta útil para deportistas, sino también en una síntesis práctica de los conocimientos adquiridos durante el ciclo de Desarrollo de Aplicaciones Multiplataforma.

4. OBJETIVOS Y ALCANCE

4.1. Objetivo general

Crear OXI, un entrenador digital inteligente para deportistas de resistencia (especialmente corredores y triatletas) que centraliza la planificación, el registro y el análisis del entrenamiento, y que es capaz de ajustar el plan de forma dinámica utilizando inteligencia artificial a partir de datos reales (historial de sesiones, sensaciones/RPE, variabilidad de la frecuencia cardiaca –VFC–, sueño y recuperación, cargas recientes y calendario personal).

4.2. Objetivos específicos

A) De producto (experiencia del deportista)

- **Planificación viva:** crear y reorganizar semanas/meses por disciplina (carrera, ciclismo, natación y fuerza) con acciones rápidas (arrastrar/soltar, duplicar, mover).
- **Registro post-entreno rico:** sensaciones (RPE), fatiga percibida, dolor/molestias, calidad del sueño, cumplimiento del objetivo y notas.
- **Resúmenes útiles:** adherencia, volumen por disciplina/semana/mes, tiempo por zonas (si el usuario las define), tendencias y alertas.
- **Trabajo “offline-first”:** uso completo sin conexión con sincronización posterior.
- **Notificaciones y rutinas:** avisos de sesiones del día y recordatorios configurables.
- **Privacidad por defecto:** datos locales y control explícito de cualquier sincronización o compartición.

B) De inteligencia artificial (IA aplicada al entrenamiento)

- **Motor de carga y riesgo:** cálculo de carga (p. ej., TRIMP/banister), monotony/strain, cociente carga aguda:crónica y alertas por picos o caídas abruptas.
- **Ajuste automático del plan:** reprogramación y modificación de volumen/intensidad en función de RPE, VFC, sueño, adherencia y eventos (exámenes/turnos).
- **Recomendador diario:** "¿Qué entreno hoy?" con alternativas equivalentes si hay fatiga, clima adverso o limitaciones de tiempo.
- **Predicción de forma y ritmos:** estimaciones de ritmos objetivos (5K–21K, personalizables) y zonas sugeridas según evolución.
- **Detección de señales de sobrecarga:** patrones que sugieren exceso de fatiga o riesgo de lesión para proponer descargas o ajustes.
- **Aprendizaje progresivo:** el sistema se adapta a cada usuario a medida que acumula datos, mejorando la personalización.
- **Opciones de despliegue:**
    - Inferencia en servidor (microservicio IA) y/o en el dispositivo (TensorFlow Lite) para funciones en tiempo real.
    - Entrenamiento/modelado en Python y exportación de modelos para uso en producción.

C) De integraciones y datos

- **Integraciones escalonadas:**
    - Google Fit para métricas básicas (pasos, tiempo activo, sueño si disponible).
    - Importación de sesiones desde archivos estándar (GPX/TCX/CSV).
    - En fases posteriores: Garmin/Strava/Coros mediante API oficiales (con OAuth).
- **Contexto del usuario:** calendario personal (bloques de estudio/trabajo), objetivos de carrera y limitaciones.
- **Esquema de datos preparado para IA:** eventos de entrenamiento, RPE, VFC, sueño, cumplimiento y métricas agregadas con sellado temporal.

D) De arquitectura y calidad (Enfoque Híbrido)

- **App Android Nativa con Arquitectura Híbrida:**
    - **Capa de Presentación (UI) en Java:** Uso de Activities, Fragments y Adapters en Java para mantener la estabilidad y aprovechar el sistema de vistas clásico de Material 3 de forma robusta.
    - **Capa de Datos y Dominio en Kotlin:** Implementación de Repositorios, Base de Datos (Room con KSP) y Red (Retrofit con Moshi) íntegramente en Kotlin. Esto permite aprovechar las Coroutines y Flow para gestionar la asincronía y el rendimiento de forma óptima.
- **Backend:** API REST principal en Java/Kotlin para sincronización multi-dispositivo y perfiles, y microservicio de IA en Python (FastAPI) para entrenamiento/inferencia cuando sea necesario.
- **Seguridad y cumplimiento RGPD:** Minimización de datos, consentimiento informado, trazabilidad de consentimientos y derecho al borrado.
- **Calidad:** Pruebas unitarias y de UI en los flujos críticos; rendimiento fluido (<2 s arranque en dispositivo medio) y estabilidad alta.

4.3. Alcance del proyecto

Incluido:

- Planificador completo (semana/mes) por disciplinas, con edición rápida y registro post-entreno detallado.
- Historial y analítica útil: adherencia, volúmenes, zonas (si procede), tendencias y panel de carga.
- Motor IA de ajuste y recomendaciones:
    - Ajuste de volumen/intensidad/orden de sesiones.
    - Sugerencias de alternativas equivalentes si hay fatiga o restricciones.
    - Predicción de ritmos/zonas y alertas por riesgo de sobrecarga.
- Recomendador diario “¿Qué entreno hoy?” con explicación de por qué se sugiere cada opción.
- Integración con Google Fit (lectura básica) e importación de GPX/TCX/CSV.
- Trabajo sin conexión con sincronización posterior y control de conflictos simple.

- Backend Java/kotlin con endpoints para usuario/sesiones/sincronización y microservicio IA en Python; opción de modelos exportados a TFLite para inferencia local.
- Privacidad por defecto, cifrado local de preferencias sensibles y controles claros de sincronización.

Fuera de alcance (no prioritario ahora, para fases avanzadas):

- Cliente iOS y versión web.
- Funcionalidades sociales avanzadas (seguidores, feed público, grupos).
- Entrenamiento de fuerza con vídeo/ejecución asistida.
- Integraciones BLE directas con sensores en tiempo real dentro de la app.

4.4. Suposiciones y dependencias

- Disponibilidad de datos del usuario (historial razonable y feedback constante) para que la IA pueda aprender y ajustar.
- Acceso y estabilidad de API externas (Google Fit, y, en fases posteriores, Garmin/Strava/Coros).
- Dispositivos Android con sensores/servicios habituales (GPS, pulso estimado vía wearables externos).
- Herramientas 100 % gratuitas en desarrollo/compilación/despliegue.
- Aceptación de políticas RGPD por parte del usuario para cualquier sincronización en la nube.

4.5. Riesgos y mitigación (resumen)

- **Datos escasos o irregulares:** la IA ofrecerá recomendaciones con “confianza” visible y pedirá feedback mínimo (RPE, sensaciones) para mejorar.
- **Sesgos o mala generalización del modelo:** validación continua, reglas de seguridad por encima de la IA y posibilidad de desactivar ajustes automáticos.
- **Cambios en API externas:** capa de integración desacoplada y contratos de datos estables.

- **Privacidad/seguridad:** minimización de datos, cifrado local y controles de sincronización; auditorías internas de permisos y registros.

4.6. Métricas de éxito (KPI)

- Adherencia ≥ 80 % tras 8 semanas de uso, con estabilidad de carga (sin picos abruptos no justificados).
- Tiempo hasta crear la primera semana de entrenos ≤ 3 minutos desde la instalación.
- Reducción de “no shows” (sesiones planificadas no realizadas) ≥ 20 % a las 6–8 semanas gracias a las recomendaciones.
- Satisfacción percibida ≥ 4/5 en encuesta interna (facilidad para planificar, utilidad de recomendaciones y claridad de explicaciones de la IA).
- Estabilidad técnica: tasa de crash < 0,5 % en sesiones de uso.

5. ESTUDIO EMPRESARIAL

5.1. La Idea: ¿Por qué nace OXI?

OXI no nace de un estudio de mercado abstracto, sino de una frustración personal. Como deportista de fondo, me encontré repitiendo el mismo ciclo una y otra vez: tenía un plan de entrenamiento perfecto en papel o en una hoja de cálculo, pero la vida real (exámenes, trabajo, un mal día de sueño) lo rompía. Las aplicaciones actuales son rígidas: si te saltas un entreno, el plan se descuadra y te sientes culpable.

Mi propuesta con OXI es simple: un entrenador que entiende que no somos robots. La aplicación no solo te dice qué hacer, sino que te pregunta "¿cómo estás?" antes y después. Si estás cansado, adapta la sesión. Si tienes poco tiempo, te da una alternativa.

No quiero competir contra gigantes como Strava en el aspecto social, ni contra Garmin en el hardware. Mi hueco está en la inteligencia de la planificación: una app que se siente como un entrenador humano que te ajusta el plan cada mañana.

5.2. Identidad Visual y Marketing

Para que OXI transmita dinamismo y tecnología, se ha diseñado una identidad visual sencilla pero reconocible.

- **Logotipo:** El imagotipo de OXI combina dos conceptos: el ciclo de movimiento (un círculo abierto) y el pulso vital (una línea de onda).

Figura 1: Imagotipo.

- **Estrategia de lanzamiento (Marketing):** Dado que el presupuesto es cero, la estrategia se basa en el crecimiento orgánico:
    - **Beta cerrada en clubes locales:** Ofrecer la app gratuita a compañeros del club de triatlón/atletismo local a cambio de feedback.
    - **ASO (App Store Optimization):** Posicionar la app en la Play Store atacando palabras clave de nicho ("planificador media maratón", "entrenador IA running") en lugar de genéricas ("correr").

5.3. Estudio de mercado y cliente objetivo

- **Alcance geográfico:** Lanzamiento en España y Latinoamérica hispanohablante (Android). Escalado posterior a otros mercados.
- **Segmentación (B2C prioritario):**
    - Corredor principiante: busca estructura y progresión segura.
    - Corredor intermedio: 3–5 sesiones/semana, quiere mejorar tiempos sin lesionarse.
    - Triatleta amateur: necesita coordinar 3 disciplinas + fuerza con poco tiempo.

    - *(Adicional a medio plazo: B2B con clubes/entrenadores para gestionar grupos).*
- **Cliente objetivo (perfil ideal):** Edad 20–45, utiliza reloj deportivo (Garmin/Coros/Suunto) y móvil Android; entrena entre 3 y 6 veces por semana; limita su tiempo por trabajo/estudios/turnos; quiere planes que se ajusten automáticamente y que expliquen las decisiones; prefiere pagar poco al mes por algo que realmente le ahorre tiempo y le dé confianza.
- **Posicionamiento:** OXI = "Tu entrenador digital que te dice qué entrenar hoy y por qué, ajustando el plan con IA según tu realidad (RPE, VFC, sueño, agenda) y manteniendo tus datos bajo control".
- **Canales de adquisición:** ASO (App Store Optimization) en Play Store; contenidos educativos (YouTube/Shorts/IG/TikTok) sobre planificación y carga; comunidades de running/Strava; partnerships con clubes locales; blog/SEO con guías prácticas (RPE, VFC, taper, etc.).
- **Modelo de precios:** Freemium: versión gratuita con funcionalidades básicas (planificación/registro) y plan Premium con IA avanzada (ajustes automáticos, recomendador diario, predicción de ritmos, alertas de carga).
    - Referencia orientativa: 5,99–7,99 €/mes o 39,99–59,99 €/año (a validar con entrevistas y test A/B).

5.4. Análisis del entorno y de la competencia: DAFO y CAME

- **Competidores de referencia:** Runna; Garmin Coach; TrainingPeaks; Strava (planes); Athletica.ai; Adidas Running; Nolio.
- **Diferenciación de OXI:** ajuste automático realmente centrado en la vida diaria del usuario + explicaciones transparentes + uso sin conexión y privacidad por defecto.

A) DAFO (Análisis interno/externo)

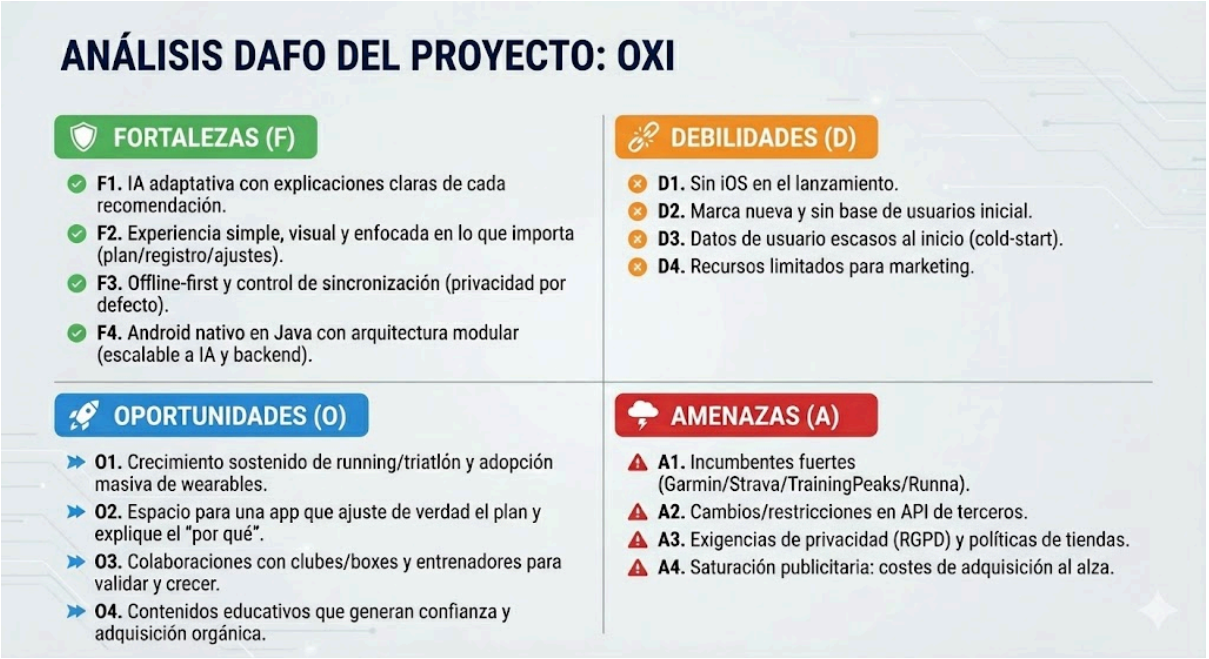

Figura 2: Análisis DAFO

B) CAME (líneas de acción)

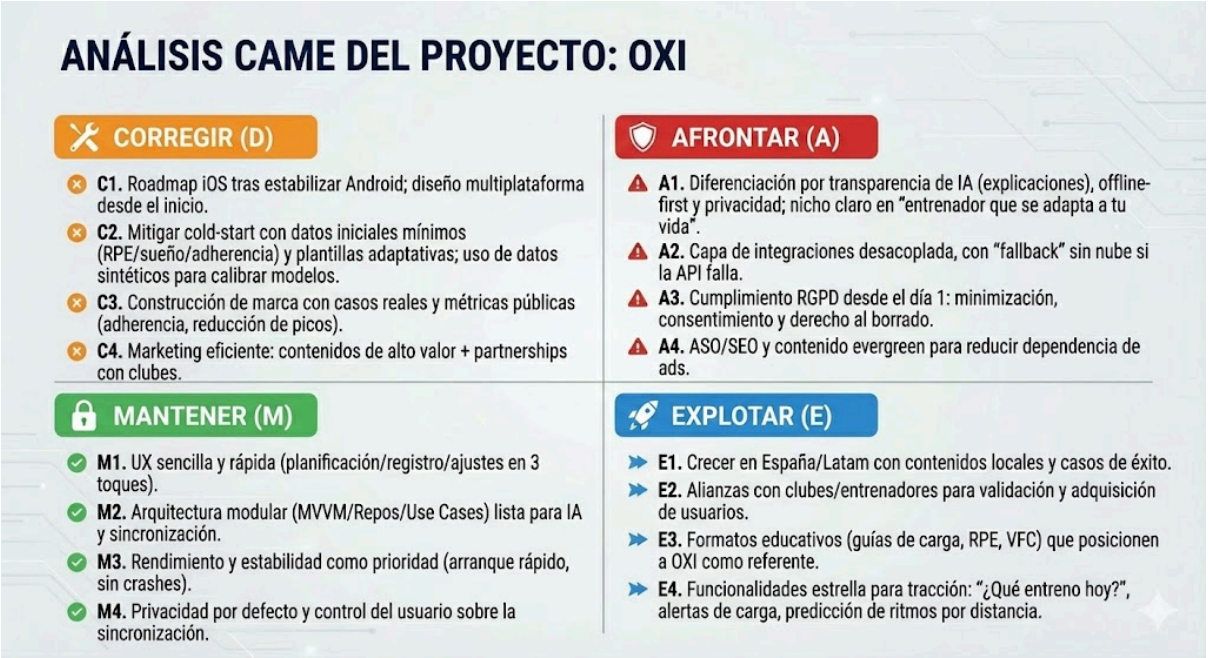

Figura 3: Análisis CAME

5.5. Ubicación

La actividad de OXI se desarrolla en régimen de teletrabajo desde Marín (Pontevedra), sin necesidad de un local físico abierto al público. La prestación del servicio es totalmente digital: la aplicación se distribuye a través de Google Play y la infraestructura de backend e inteligencia artificial se aloja en servidores en la nube.

Esta ubicación se justifica por:

- **Reducción de costes fijos:** se evita el alquiler o compra de un local, pudiendo destinar más recursos al desarrollo y mejora del producto.
- **Flexibilidad operativa:** el teletrabajo permite adaptar horarios a la carga de trabajo y compatibilizar el proyecto con otras actividades.
- **Escalabilidad:** la infraestructura técnica (backend, bases de datos, microservicios de IA) se despliega en la nube, permitiendo crecer en número de usuarios sin depender de una ubicación física concreta.
- **Acceso a talento externo:** el modelo remoto facilita la colaboración puntual con diseñadores, especialistas en IA o QA sin necesidad de compartir espacio físico.

En caso de necesitar reuniones presenciales (entrevistas con usuarios, pruebas de usabilidad, reuniones comerciales), se recurriría a espacios de coworking en la zona de Pontevedra/Vigo o a salas puntuales de alquiler, evitando compromisos de alquiler a largo plazo.

5.6. Forma jurídica y obligaciones legales

Forma jurídica elegida (fase inicial): empresario individual (trabajador autónomo).

Justificación:

- **Simplicidad administrativa:** el alta como autónomo es rápida y con trámites relativamente sencillos.
- **Coste reducido:** no requiere capital social inicial ni gastos notariales o registrales significativos.
- **Flexibilidad:** permite iniciar la actividad, validar el modelo de negocio y, en función de la evolución, valorar la creación posterior de una sociedad.

Posible evolución futura: creación de una Sociedad de Responsabilidad Limitada (SL) cuando:

- Exista necesidad de incorporar socios, inversores o empleados.
- El volumen de facturación y el riesgo operativo hagan aconsejable separar el patrimonio personal del empresarial.
- Se quiera mejorar la imagen de solvencia frente a clientes B2B (clubs, empresas, etc.).

Obligaciones legales básicas como autónomo:

- **Alta fiscal y en la Seguridad Social:**
    - Alta censal en la Agencia Tributaria (modelo 036/037) en el epígrafe de actividad relacionado con programación y consultoría informática.
    - Alta en el Régimen Especial de Trabajadores Autónomos (RETA) y pago de la cuota mensual correspondiente (considerando posibles bonificaciones por inicio de actividad).
- **Obligaciones fiscales:**
    - IVA trimestral (modelo 303) y resumen anual (modelo 390), si la actividad está sujeta y no exenta.
    - Pagos fraccionados del IRPF (modelo 130) en caso de no estar sometido de forma general a retención en las facturas.
    - En caso de contratar servicios profesionales a otros autónomos o alquilar un local, gestión de retenciones e ingresos a cuenta (modelos 111/190, 115/180).
- **Obligaciones contables y registrales:**
    - Llevanza de libros registro de ingresos y gastos, inversión y bienes de inversión.
    - Conservación de facturas emitidas y recibidas durante los plazos legales.
- **Cumplimiento normativo específico del servicio digital:**
    - **Protección de datos (RGPD y LOPDGDD):**
        - Elaboración de un registro de actividades de tratamiento (usuarios de la app).

        - Determinación de la base jurídica (principalmente ejecución de contrato y consentimiento).
        - Información clara al usuario mediante política de privacidad y textos legales en la app y en la ficha de Google Play.
        - Medidas técnicas y organizativas: minimización de datos, cifrado de información sensible, control de accesos, copias de seguridad.
        - Garantía de derechos de acceso, rectificación, supresión, limitación, portabilidad y oposición.
    - **Servicios de la sociedad de la información (LSSI-CE):**
        - Aviso legal, condiciones de uso y, en su caso, política de cookies.
        - Identificación del titular del servicio y medios de contacto.
        - Cumplimiento de las normas sobre comunicaciones comerciales electrónicas.
    - **Propiedad intelectual e industrial:**
        - Protección del nombre comercial y, en su caso, registro de la marca “OXI”.
        - Gestión correcta de licencias de las librerías y herramientas de software utilizadas, evitando incompatibilidades.

5.7. Recursos humanos y producción

En la fase inicial del proyecto, la empresa está formada por un único promotor que asume varios roles:

- Dirección y gestión del proyecto.
- Análisis y diseño funcional de la aplicación.
- Desarrollo de la aplicación Android (Java, MVVM, Room, Material 3).
- Diseño e implementación del backend (Java/kotlin) y de los primeros microservicios de IA (Python/FastAPI).
- Pruebas básicas de funcionamiento (testing funcional) y corrección de errores.
- Publicación, mantenimiento en Google Play y atención al usuario.

Proceso de producción:

- Planificación del desarrollo mediante sprints cortos, con un listado de tareas priorizadas (backlog).
- Uso de sistemas de control de versiones (Git) para gestionar el código fuente y las distintas versiones del producto.
- Integración continua básica: compilación, ejecución de tests y generación de versiones de prueba (APK) antes de subir nuevas versiones a la tienda.
- Ciclo de mejora continua: recopilación de feedback de usuarios, análisis de métricas (retención, errores, uso de funcionalidades) y planificación de nuevas versiones.

A medio plazo, si la aplicación evoluciona favorablemente, se considerará la colaboración puntual con otros perfiles (diseño UX/UI, especialista en datos/IA, testing), manteniendo un núcleo muy reducido y flexible.

5.7.1. Organigrama

Dado que el proyecto se inicia con un solo promotor, el organigrama es muy simple. A nivel gráfico, la estructura puede representarse así: Promotor / Director del proyecto ↓ Desarrollo y mantenimiento del producto (Android + backend + IA) ↓ Soporte y atención al usuario

En fases posteriores, si se incorporan colaboradores, la estructura podría evolucionar a:

- Dirección de proyecto.
- Área técnica (desarrollo Android, backend y sistemas).
- Área de datos e IA.
- Área de diseño/UX.
- Área de soporte y comunicación con el usuario.

5.7.2. Convenio colectivo

En el caso de que más adelante se contrate personal por cuenta ajena, la empresa se acogería al convenio colectivo de referencia del sector, por ejemplo:

- Convenio colectivo estatal de empresas de consultoría y estudios de mercado y de la opinión pública.

- O, en su caso, al convenio que corresponda al sector de servicios informáticos y consultoría tecnológica vigente en el momento de la contratación.

5.7.3. Prevención de riesgos laborales

Modalidad preventiva: Considerando el reducido tamaño de la empresa y que la actividad se realiza principalmente en régimen de teletrabajo, la modalidad de organización preventiva más adecuada es la contratación de un servicio de prevención ajeno que asesore en materia de seguridad y salud laboral. Además, el propio promotor asume la responsabilidad de integrar la prevención en la organización de la actividad diaria.

Se han identificado los riesgos principales asociados al puesto (ergonómicos, psicosociales, eléctricos y del entorno doméstico) y se han establecido sus correspondientes medidas correctoras. El detalle completo de la evaluación de riesgos y el plan de medidas de prevención y protección se encuentra disponible en el **Anexo I: Prevención de Riesgos Laborales**.

5.7.4. Recursos materiales y humanos necesarios

Recursos humanos:

- **Fase inicial:**
  - Un promotor/desarrollador que asume las funciones de análisis, diseño, programación, pruebas básicas, publicación y soporte.
- **Fase de crecimiento (a valorar):**
  - Un desarrollador adicional (Android o backend) para acelerar el roadmap.
  - Un especialista en datos e inteligencia artificial para entrenar y mejorar modelos.
  - Colaboración puntual en diseño gráfico/UX.
  - Soporte al usuario (a tiempo parcial) si aumenta la base de usuarios.

Recursos materiales:

- **Equipos informáticos:**

  - Ordenador de sobremesa o portátil de gama media/alta para desarrollo (suficiente RAM y CPU).
  - Dispositivo(s) Android físico para pruebas reales (móvil principal y, si es posible, uno adicional de gama media-baja).
  - Periféricos: monitor externo, teclado, ratón, auriculares para reuniones.
- **Software y herramientas:**
  - IDE de desarrollo Android (Android Studio) y herramientas gratuitas asociadas (SDK, emuladores).
  - Entorno de desarrollo para backend y microservicios de IA (Java, Python, FastAPI u otros frameworks, editores de código, etc.).
  - Sistemas de control de versiones (Git) y repositorio remoto (GitHub u otro).
  - Herramientas de comunicación y gestión de tareas (correo, videollamadas, gestor de proyectos).
- **Infraestructura técnica:**
  - Alojamiento en la nube para el backend y posibles microservicios de IA (servidor virtual o plataforma como servicio).
  - Bases de datos en la nube para sincronización y almacenamiento de información (con copias de seguridad periódicas).
  - Servicios auxiliares (en su caso): envío de correos transaccionales, analítica de uso, monitorización básica.

5.8. Plan económico-financiero

El plan económico-financiero tiene como objetivo estimar las necesidades de inversión y gastos, las posibles fuentes de financiación y una previsión de ingresos para el primer año de actividad.

5.8.1. Inversiones y gastos iniciales (aproximación)

Inversiones iniciales:

- Equipos informáticos (ordenador y periféricos específicos para desarrollo) — si no se dispusiera ya de ellos.
- Dispositivo móvil Android adicional para pruebas.
- Registro de nombre de dominio y alojamiento web básico.

- Posible registro de marca y asesoramiento legal inicial.

Gastos iniciales estimados:

- Renovación/mejora de equipo informático: 1.000 €.
- Dispositivo Android de prueba: 250 €.
- Dominio y hosting web (1 año): 80 €.
- Asesoría legal/básica (textos legales, alta de actividad): 300 €.
- Registro de marca (si se realiza): 150–200 €.
- **Total inversiones iniciales aproximadas: alrededor de 1.600–1.800 €.**

5.8.2. Gastos fijos y variables anuales

Gastos fijos anuales (orientativos):

- Cuota de autónomo (considerando posibles bonificaciones iniciales): 1.200–2.000 €/año (según tramo y bonificaciones).
- Alojamiento backend y bases de datos en la nube: 300–500 €/año (según consumo).
- Dominio y hosting web: 80–100 €/año.
- Herramientas y servicios de apoyo (almacenamiento en la nube, herramientas de analítica, etc.): 100–200 €/año.
- Asesoría contable y fiscal (si se contrata): 600–900 €/año.

Gastos variables:

- **Publicidad y Marketing:** Se estima una inversión necesaria de 600€ – 1.000€ anuales (Ads en redes sociales y buscadores) para la captación inicial de usuarios y validación de mercado.
- Posibles comisiones de pasarelas de pago o plataformas de cobro.
- Costes de soporte externo o colaboraciones puntuales (diseño, IA, testing).

5.8.3. Fuentes de financiación, ayudas y subvenciones

Fuentes de financiación:

- Aportación propia del promotor (ahorro personal) para cubrir inversiones iniciales y gastos de los primeros meses.
- Posibles microcréditos para emprendedores si fuera necesario disponer de liquidez adicional.

- Reinversión de los primeros ingresos obtenidos por suscripciones o servicios relacionados.

Ayudas y subvenciones (a valorar):

- Programas de apoyo al autoempleo y emprendimiento juvenil/tecnológico convocados por administraciones públicas (por ejemplo, ayudas al inicio de actividad de la Xunta de Galicia o del SEPE, según requisitos vigentes).
- Subvenciones específicas para proyectos tecnológicos o de transformación digital.
- Programas de asesoramiento y mentorización ofrecidos por entidades públicas o privadas (viveros de empresas, cámaras de comercio, etc.).

5.8.4. Estimación de ingresos a un año

La estimación de ingresos del primer año debe ser prudente, ya que se trata de una fase inicial de lanzamiento y validación del producto.

Ejemplo de escenario conservador:

- **Modelo freemium:** mayoría de usuarios gratuitos y una parte menor de usuarios que contratan la versión de pago con funcionalidades avanzadas de IA.
- **Suposición:**
    - 300 usuarios registrados al final del primer año.
    - Un 20 % convierte a versión de pago (60 usuarios de pago).
    - Precio medio mensual del plan de pago: 6 €/mes.
    - Tiempo medio de suscripción efectiva durante el primer año: 8 meses.

Ingresos aproximados: 60 usuarios × 6 €/mes × 8 meses = 2.880 €.

A estos ingresos por suscripciones se podrían sumar, en el futuro, ingresos adicionales derivados de:

- Colaboraciones con clubes o entidades deportivas (modelos B2B).
- Servicios adicionales de análisis y planificación personalizada.

En cualquier caso, durante el primer año, el objetivo principal no es tanto maximizar el beneficio económico como:

- Validar que el producto resuelve problemas reales para los deportistas.
- Alcanzar una base suficiente de usuarios activos.

- Obtener datos y feedback que permitan mejorar el motor de IA y la experiencia de uso.

6. ESTUDIO TÉCNICO

6.1. Análisis y planificación

OXI se ha diseñado como un proyecto modular y escalable, con una arquitectura clara que separa la interfaz de usuario, la lógica de negocio, el acceso a datos y, en fases posteriores, los servicios de inteligencia artificial y sincronización en la nube.

Estructura general del proyecto:

- **Capa de presentación (Android):**
  - Actividades y fragments que implementan las pantallas principales (onboarding, planificación, detalle de sesión, resumen, ajustes).
  - ViewModels (patrón MVVM) que gestionan el estado de la UI y coordinan la interacción con los casos de uso.
- **Capa de dominio:**
  - Casos de uso (use cases) que encapsulan la lógica de negocio: creación de sesiones, actualización de entrenamientos, cálculo de métricas básicas, obtención de resúmenes, etc.
  - Modelos de dominio independientes de la infraestructura (entidades lógicas de usuario, sesión, feedback, métricas).
- **Capa de datos:**
  - Repositorios que abstraen el origen de los datos (base de datos local con Room, configuración local, y futuro backend remoto).
  - Implementación de Room (entidades, DAOs, base de datos) para almacenar sesiones, feedback, configuraciones y métricas agregadas.
- **Capa de servicios externos:**
  - Diseño de un backend en Java/kotlin (API REST) para sincronización y gestión de perfiles multi-dispositivo (pendiente de completar o en fase de prototipo).

    - Microservicio de IA en Python para recomendaciones y análisis avanzado, pensado para ejecutarse en servidor o para exportar modelos a TensorFlow Lite.

Metodología: Se ha seguido una metodología iterativa-incremental, inspirada en Scrum pero adaptada a un proyecto con un único promotor:

- **Iteraciones cortas (sprints)** centradas en objetivos claros: primero estructura de la app y navegación básica, luego persistencia y lógica de negocio, posteriormente métricas y, por último, integración inicial de IA.
- **Revisión al final de cada iteración:**
    - Comprobación del funcionamiento real en dispositivo físico.
    - Revisión del diseño de pantallas (claridad, número de pasos, tiempo para completar acciones).
    - Ajuste del backlog en función de dificultades técnicas y de la experiencia de uso.

Priorización:

1. Primero, funcionalidades críticas para el uso diario (planificación, registro, historial).
2. Después, métricas y resúmenes.
3. Finalmente, integración de IA y sincronización con servicios externos.

Planificación de trabajo:

El proyecto sigue una estrategia de desarrollo incremental de 6 meses dividida en fases de Análisis, Core Dev, UI, Lógica IA, Testing y Defensa. El diagrama de Gantt detallado con los hitos temporales y la explicación de la distribución de cargas de trabajo se adjunta en el **Anexo III: Cronograma del Proyecto**.

6.2. Requisitos hardware y software

Para el desarrollo del proyecto se ha configurado un entorno de trabajo profesional estándar para desarrollo Android nativo. Se ha utilizado un equipo informático de alto rendimiento y dispositivos físicos para las pruebas reales.

En cuanto al software, se ha trabajado con la última versión estable de Android Studio, utilizando Java 17 y el SDK 35, integrando librerías modernas como AndroidX, Material 3 y Room. La infraestructura de backend y los microservicios de

IA se han diseñado para ser desplegados en entornos Linux/Cloud. El listado exhaustivo de especificaciones técnicas, versiones exactas, librerías y requisitos para el usuario final se detalla en el **Anexo II: Ficha Técnica del Entorno**.

6.3. Implementación

Lenguajes y librerías empleadas:

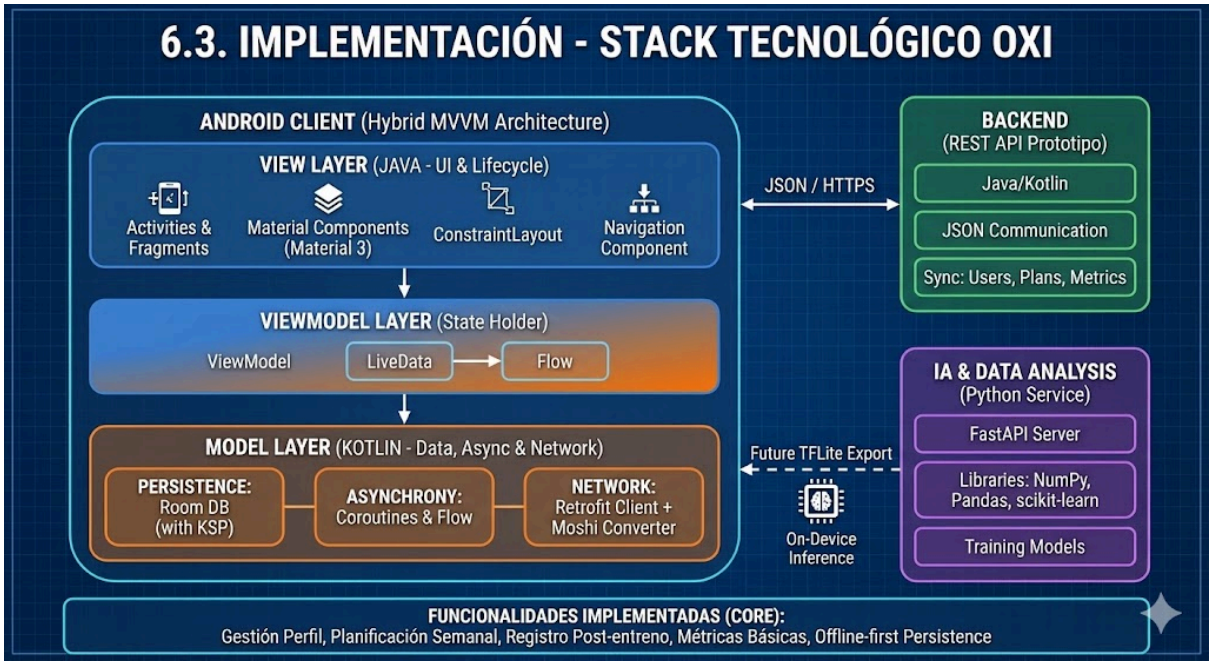

Figura 5: Stack tecnológico y arquitectura del sistema.

Funcionalidades implementadas:

- Gestión de perfil básico del usuario.
- Planificación semanal de entrenamientos (Alta, edición y eliminación).
- Registro post-entreno (RPE, fatiga, notas).
- Historial y métricas básicas (Volumen, adherencia).
- Persistencia local completa (Offline-first).

6.4. Funcionamiento general de la aplicación

El flujo principal de la aplicación se puede resumir en los siguientes pasos:

Figura 6: Flujo de usuario principal.

6.5. Interfaces de la aplicación

El diseño sigue estrictamente los principios de Material Design 3, priorizando la legibilidad en movimiento (botones grandes, alto contraste). A continuación se muestran los flujos principales:

Flujo 1: Planificación Semanal La pantalla principal ofrece una vista de "Semana Vista". El usuario puede ver de un vistazo la carga total. Al mantener pulsado un entrenamiento, se activa el modo edición rápida (Drag & Drop) para mover sesiones si surge un imprevisto.

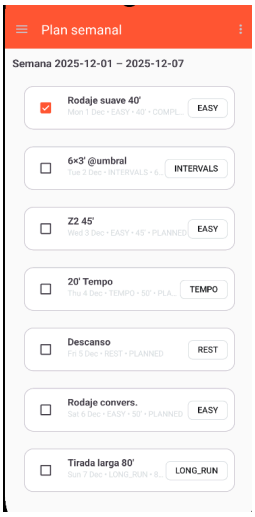

Figura 7: Vista de planificación semanal.

Flujo 2: El registro post-entreno (Feedback) Al terminar una sesión, no solo importa el "check" de realizado. La app despliega un bottom-sheet donde el usuario introduce su RPE (Esfuerzo Percibido) deslizando una barra. Este dato es el que alimenta al algoritmo para las futuras recomendaciones.

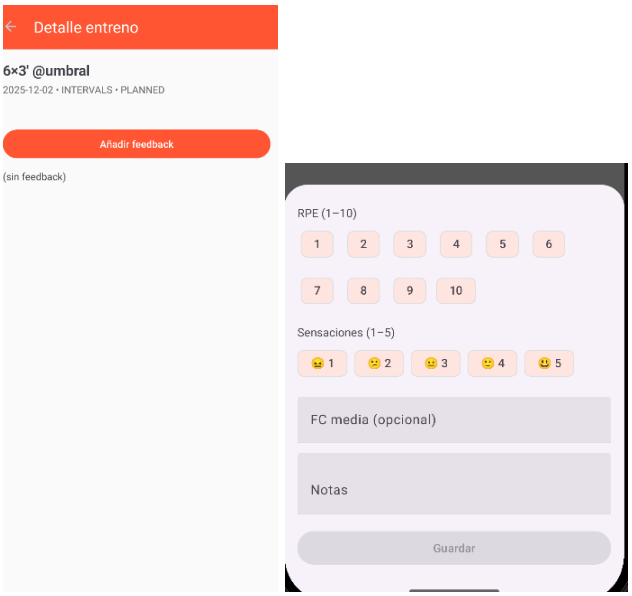

Figura 8: Pantalla de registro de feedback.

6.6. Pruebas y validación

Estrategia de pruebas:

- **Pruebas unitarias:**
  - Pruebas en casos de uso clave (creación de sesiones, cálculo de adherencia, actualización de métricas).
  - Verificación de la lógica de negocio sin depender de la interfaz.
- **Pruebas de integración:**
  - Comprobación del correcto funcionamiento entre ViewModels, repositorios y base de datos Room.
  - Pruebas de inserción, actualización y lectura de datos en la base local.
- **Pruebas de interfaz (instrumentadas):**
  - Navegación entre pantallas.
  - Creación, edición y eliminación de sesiones desde el punto de vista del usuario.

- **Pruebas de rendimiento:**
  - Tiempo de apertura de la app.
  - Fluidez al moverse entre vistas y al cargar listas de sesiones.
- **Pruebas de usuario (validación):**
  - Entrega de versiones de prueba a usuarios cercanos (corredores/triatletas).
  - Recogida de feedback sobre:
    - Claridad de la interfaz.
    - Tiempo necesario para crear una semana de entrenamientos.
    - Utilidad del registro post-entreno y de los resúmenes.

Validación: El objetivo es que un usuario pueda:

1. Configurar su perfil.
2. Planificar una semana completa en pocos minutos.
3. Registrar fácilmente sus sesiones y consultar sus avances. Se analiza si la aplicación realmente reduce la fricción de planificación y mejora la claridad sobre la carga semanal.

6.7. Implantación y despliegue

Escenarios de despliegue:

- **Desarrollo y pruebas internas:**
  - Generación de APK de prueba (debug/release) para instalarlas manualmente en dispositivos propios.
  - Uso de canales internos de distribución (por ejemplo, enlaces directos o servicios de test).
- **Despliegue en Google Play:**
  - Creación de la ficha de la aplicación (nombre, descripción, capturas, política de privacidad).
  - Uso de pistas de lanzamiento:
    - **Internal testing:** para pruebas internas rápidas.
    - **Closed testing:** con un grupo limitado de usuarios.
    - **Production:** publicación abierta al público una vez superadas las fases anteriores.

Gestión de versiones:

- Incremento de versionCode y versionName en cada actualización.
- Notas de versión indicando cambios, mejoras y correcciones.

Implantación:

- **Plan de puesta en marcha:**
  - Publicar una primera versión funcional centrada en planificación y registro.
  - Monitorizar errores, bloqueos y métricas de uso básico.
  - Recoger feedback de usuarios para priorizar mejoras e implementar la parte de IA progresivamente.
- **Evolución:**
  - Despliegue gradual de nuevas funcionalidades, empezando por mejoras en resúmenes y paneles de carga, y posteriormente introduciendo el motor de recomendaciones con IA.
  - Mantenimiento de compatibilidad de datos mediante migraciones controladas de Room y del backend.

6.8. Medidas de seguridad y protección de datos

Medidas técnicas en la aplicación:

- **Minimización de datos:**
  - Se almacenan únicamente los datos necesarios para el funcionamiento de la app (perfil básico, sesiones, registros, métricas).
  - Se evita pedir información que no aporte valor directo (por ejemplo, datos personales excesivos).
- **Seguridad local:**
  - Uso de la base de datos local (Room) con acceso controlado desde la propia app.
  - Posibilidad de cifrado del almacenamiento de preferencias sensibles (p. ej., con EncryptedSharedPreferences).
- **Comunicación segura:**
  - Cuando se active la sincronización con un backend, uso obligatorio de HTTPS para la transmisión de datos.

    - Validación básica de certificados y protección frente a ataques de intermediario (MITM).
- **Acceso y autenticación:**
    - Si se utiliza registro de usuario, se realiza con credenciales gestionadas de forma segura (las contraseñas no se almacenan en texto plano en el servidor, sino que se emplean técnicas de hash y sal).
    - Gestión de tokens de sesión, evitando exponer información sensible en el lado del cliente.

Protección de datos (RGPD y LOPDGDD):

- **Base jurídica:**
    - El tratamiento de datos se realiza principalmente para la prestación del servicio (ejecución del contrato con el usuario) y, en su caso, con el consentimiento del usuario para funcionalidades adicionales (por ejemplo, analítica opcional, sincronización en la nube).
- **Información al usuario:**
    - La aplicación incluye una política de privacidad clara y accesible, así como un aviso legal y condiciones de uso.
    - Se explica qué datos se recopilan, con qué finalidad, durante cuánto tiempo y con qué derechos cuenta el usuario.
- **Derechos del usuario:**
    - Se prevé un mecanismo para ejercer los derechos de acceso, rectificación, supresión, portabilidad y oposición (por ejemplo, a través de un correo de contacto o desde la propia app).
    - Posibilidad de solicitar el borrado completo de la cuenta y de todos los datos asociados en el servidor.
- **Relación con terceros:**
    - En caso de integrarse servicios externos (por ejemplo, Google Fit), se respetan las políticas de dichos servicios y se solicita autorización expresa al usuario.

    - Se firman los contratos necesarios con proveedores que traten datos por cuenta del responsable (alojamiento, analítica, etc.), actuando como encargados del tratamiento.

Con todo ello, se busca que OXI no solo sea una herramienta útil para el deportista, sino también una aplicación segura y respetuosa con la privacidad, alineada con la normativa vigente en materia de protección de datos.

7. PROPUESTAS DE MEJORA

A lo largo del desarrollo de OXI han ido apareciendo múltiples ideas de mejora, tanto a nivel técnico como a nivel de producto y de modelo de negocio. A continuación, se recogen las más relevantes, planteadas como una hoja de ruta futura.

7.1. Propuestas de mejora técnicas

- **Desarrollo de backend completo y sincronización en la nube:** Actualmente la aplicación está centrada en el funcionamiento local (offline-first) con una base de datos en el dispositivo. Una mejora clave sería completar el backend en Java y añadir un sistema de sincronización opcional que permita:
    - Usar OXI en varios dispositivos manteniendo los datos actualizados.
    - Realizar copias de seguridad automáticas en la nube.
    - Facilitar la recuperación de datos en caso de pérdida o cambio de móvil.
- **Integración real con servicios externos (Google Fit, Garmin, Strava, etc.):** Aunque el diseño ya contempla la integración con servicios de terceros, la implementación efectiva de estas conexiones permitiría:
    - Importar automáticamente entrenamientos registrados por el reloj deportivo.
    - Obtener datos adicionales (FC, VO_2 estimado, sueño, pasos, etc.).
    - Evitar registros manuales, reduciendo errores y mejorando la experiencia de uso.

- **Motor de IA más avanzado y ejecución híbrida (nube + dispositivo):** La versión inicial se centra en preparar los datos y definir reglas básicas. Una mejora importante consistiría en:
    - Entrenar modelos de machine learning con datos reales de usuarios (respetando siempre la privacidad).
    - Implementar modelos específicos por tipo de deportista (corredor, triatleta, nivel, historial de lesiones).
    - Combinar inferencia en servidor y en el dispositivo (TensorFlow Lite) para mantener recomendaciones incluso sin conexión.
- **Paneles de métricas más completos:** Ampliar la parte de analítica para mostrar:
    - Carga aguda y crónica, monotony y strain.
    - Alertas visuales de riesgo de sobrecarga o de falta de estímulo.
    - Evolución de ritmos y tiempos en distintas distancias.
- **Tests automatizados más extensos:** Otra mejora importante sería aumentar la cobertura de pruebas:
    - Más pruebas unitarias y de integración.
    - Pruebas automatizadas de UI para flujos críticos (planificación, registro, borrado de sesiones).
    - Integración continua con pipelines más avanzados (build, test, informes de cobertura).

7.2. Propuestas de mejora de producto y negocio

- **Publicación internacional y localización:** Traducir la aplicación al inglés y a otros idiomas estratégicos permitiría ampliar el mercado objetivo, empezando por la Unión Europea y Latinoamérica.
- **Versión iOS:** Aunque el foco actual es Android, una versión para iOS permitiría:
    - Llegar a un segmento importante de usuarios.
    - Posicionarse como solución multiplataforma real.

- **Planes específicos y plantillas inteligentes:** Incorporar plantillas de planes de entrenamiento (10K, media maratón, maratón, triatlón…) que la IA pueda adaptar automáticamente en función de:
    - Fecha objetivo.
    - Nivel de partida.
    - Disponibilidad semanal.
- **Modo entrenador/club (B2B):** Diseñar una versión orientada a entrenadores y clubes deportivos, en la que:
    - Un entrenador pueda gestionar a varios deportistas.
    - Se puedan asignar planes, revisar adherencia y ajustar la carga desde un panel central.
- **Estrategia de monetización más detallada:** Refinar el modelo freemium para:
    - Definir claramente qué funcionalidades son gratuitas y cuáles son de pago.
    - Ajustar precios y planes en función del tipo de usuario (individual, club, etc.).

8. DIFICULTADES ENCONTRADAS

Durante el desarrollo del proyecto se han presentado diversas dificultades que han influido en la planificación y en la forma de abordar el trabajo. Las más significativas han sido las siguientes:

8.1. Definición del alcance y riesgo de sobredimensionar el proyecto

Una de las primeras dificultades fue decidir hasta dónde llegar con la primera versión de OXI. El principal riesgo inicial fue la definición del alcance (scope creep), intentando abarcar funcionalidades complejas... lo que podía convertir el proyecto en algo inabarcable dentro del tiempo disponible.

Para solucionarlo, fue necesario:

- Priorizar las funcionalidades clave (planificación, registro y métricas básicas).

- Separar claramente lo que debía estar implementado en esta fase de lo que formaría parte de la evolución futura.
- Mantener la visión global ambiciosa, pero con un alcance técnico realista para la entrega del proyecto.

8.2. Interoperabilidad Java-Kotlin y curva de aprendizaje

Aunque ya contaba con conocimientos previos de programación, la decisión de utilizar una arquitectura híbrida presentó retos técnicos específicos más allá de la arquitectura MVVM estándar.

La principal dificultad radicó en la interoperabilidad entre Java y Kotlin, específicamente en cómo consumir las suspend functions y los Flows de la capa de datos (Kotlin) desde la capa de vista (Java). Fue necesario:

- Estudiar profundamente cómo la JVM maneja ambos lenguajes.
- Implementar adaptadores o convertir los Flows a LiveData para que las clases en Java pudieran observar los cambios de datos sin problemas de concurrencia.
- Organizar el código en capas estrictas para evitar referencias circulares o dependencias incorrectas entre los módulos de Java y Kotlin.

El resultado es un código base moderno en su núcleo y estable en su superficie, pero requirió un esfuerzo de refactorización significativo.

8.3. Diseño de la estructura de datos pensando en la IA

Otra dificultad importante fue pensar la base de datos no solo para la app actual, sino también como fuente de datos para futuros modelos de inteligencia artificial. Esto obligó a replantear:

- Qué campos guardar (RPE, adherencia, cargas semanales, etc.).
- Cómo relacionar entrenamientos, feedback, métricas y objetivos.
- Cómo dejar margen para añadir nuevos datos sin romper la estructura.

La forma de resolverlo fue:

- Dibujar esquemas y diagramas de entidad-relación con distintas opciones.

- Simplificar estructuras demasiado complejas y dejar campos/extensiones preparados para el futuro.

8.4. Gestión del tiempo y compatibilidad con otras obligaciones

Compatibilizar el desarrollo de OXI con estudios y otras responsabilidades personales no ha sido sencillo. El riesgo era avanzar de forma irregular o posponer tareas importantes.

Se intentó solucionar:

- Estableciendo bloques de tiempo dedicados exclusivamente al proyecto.
- Dividiendo las tareas en subtareas más pequeñas y alcanzables.
- Revisando periódicamente el progreso y ajustando el plan de trabajo.

9. CONCLUSIONES

El desarrollo de OXI ha exigido resolver problemas complejos de arquitectura, específicamente la convivencia entre el sistema de vistas clásico de Android y la lógica asíncrona moderna en Kotlin. Ha sido un ejercicio completo en el que he tenido que combinar:

- **Análisis y diseño:** definir un problema real de deportistas de resistencia y traducirlo en requisitos, funcionalidades y flujos de trabajo.
- **Arquitectura y desarrollo:** aplicar patrones como MVVM, usar Room, Navigation y Material 3, y organizar el proyecto de forma modular para permitir su evolución.
- **Visión de producto:** pensar no solo en lo que se puede programar, sino en lo que realmente aporta valor al usuario.
- **Perspectiva empresarial:** elaborar un estudio de mercado, analizar la competencia, plantear una propuesta de valor diferenciada y definir un modelo de negocio inicial.

Entre las principales aprendizajes destacaría:

- La importancia de limitar el alcance inicial sin renunciar a una visión ambiciosa a largo plazo.

- El valor de una buena arquitectura desde el principio, que permite añadir funcionalidades y evolucionar el proyecto con menos fricción.
- La necesidad de pensar en datos y métricas desde el diseño, especialmente si se quiere introducir inteligencia artificial en fases posteriores.
- La relación entre la parte técnica y la parte empresarial: una buena idea tecnológica pierde fuerza si no se explica bien al usuario y al mercado.

Como resultado, OXI cuenta ahora con una arquitectura escalable que permitirá la integración progresiva de los módulos de IA sin comprometer la estabilidad de la aplicación hacia un entrenador digital verdaderamente inteligente, capaz de adaptar el entrenamiento a la vida real del deportista. El proyecto no está “cerrado”; al contrario, esta primera versión es el punto de partida para seguir aprendiendo, mejorando y ampliando tanto las capacidades técnicas como la madurez del producto.

10. BIBLIOGRAFÍA Y RECURSOS

A lo largo del desarrollo del proyecto se han consultado diferentes recursos, tanto técnicos como relacionados con el área de negocio y la normativa aplicable. Entre ellos destacan:

10.1. Documentación técnica y desarrollo Android

- **Android Developers - Guía de arquitectura de aplicaciones (MVVM):** https://developer.android.com/topic/architecture
- **Android Developers - Interoperabilidad de Java y Kotlin:** https://developer.android.com/kotlin/interop
- **Material Design 3 - Componentes y directrices de diseño:** https://m3.material.io/
- **Entrenamiento de datos con Room (Android Developers):** https://developer.android.com/training/data-storage/room
- **Documentación oficial de Kotlin (Coroutines y Flow):** https://kotlinlang.org/docs/coroutines-overview.html

10.2. Inteligencia artificial y análisis de datos

- **TensorFlow Lite para dispositivos móviles:** https://www.tensorflow.org/lite/android
- **Scikit-learn: Machine Learning en Python:** https://scikit-learn.org/stable/user_guide.html
- **FastAPI - Documentación oficial:** https://fastapi.tiangolo.com/
- **Pandas - Análisis de datos en Python:** https://pandas.pydata.org/docs/

10.3. Negocio, mercado y emprendimiento

- **Google Play Console - Ficha de la tienda y ASO:** https://play.google.com/console/about/
- **Strava Developers (Referencia de API y mercado):** https://developers.strava.com/
- **Google Fit REST API:** https://developers.google.com/fit/rest

10.4. Normativa y protección de datos

- **Agencia Española de Protección de Datos (AEPD) - Guías para desarrolladores:** https://www.aepd.es/es/areas-de-actuacion/proteccion-desde-el-diseno
- **BOE - Ley Orgánica 3/2018, de Protección de Datos Personales (LOPDGDD):** https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673
- **Reglamento General de Protección de Datos (RGPD - UE):** https://gdpr.eu/

Esta combinación de recursos ha permitido abordar el proyecto desde una perspectiva técnica, de producto y legal, y ha contribuido de forma significativa a la calidad y coherencia del resultado final.

11. ANEXOS

Anexo I: Prevención de Riesgos Laborales

Principales riesgos identificados en el puesto (teletrabajo en informática):

- **Riesgos ergonómicos:** Posturas mantenidas y forzadas (cuello, espalda, muñecas) y uso prolongado de pantallas de visualización de datos.

- **Riesgos psicosociales:** Carga mental elevada, plazos de entrega, dificultad para desconectar y aislamiento social.
- **Riesgos eléctricos:** Uso de equipos eléctricos conectados a la red.
- **Riesgos asociados al entorno doméstico:** Caídas al mismo nivel e iluminación inadecuada.

Medidas de prevención y protección:

- **Ergonomía:** Silla regulable, mesa adecuada, pantalla a la altura de los ojos, periféricos independientes y descansos breves frecuentes (regla 20-20-20).
- **Organización del trabajo:** Horarios razonables, planificación de tareas, evitar jornadas excesivas y separación clara entre tiempo de trabajo y descanso.
- **Seguridad eléctrica:** Uso de regletas homologadas, revisión periódica de cables y evitar sobrecargas.
- **Entorno:** Mantener el área ordenada y asegurar iluminación adecuada (luz natural + artificial regulada).

Anexo II: Ficha Técnica del Entorno

Requisitos de hardware (desarrollo):

- Ordenador de sobremesa/portátil con procesador multinúcleo, 16 GB RAM y 512 GB SSD.
- Dispositivo físico Android (gama media, Android 10+) y dispositivo opcional gama baja para pruebas de estrés.

Requisitos de software (desarrollo):

- **SO:** Windows/Linux/macOS.
- **IDE:** Android Studio (compatible compileSdk 35).
- **JDK:** Java 17.
- **SDK Android:** compileSdk 35, targetSdk 34, minSdk 24.
- **Control de versiones:** Git (GitHub). Repositorio disponible en: https://github.com/alvarocastisoto/oxi

Stack Tecnológico (Librerías y Frameworks):

- **Lenguajes:** Java/Kotlin.

- **UI:** AndroidX, Material 3.
- **Persistencia:** Room.
- **Arquitectura:** Navigation Component, ViewModel, LiveData/Flow.
- **Backend/IA:** API REST (Java/Spring Boot), Microservicio IA (Python/FastAPI), Despliegue PaaS (Linux/JVM).

Requisitos para el usuario final:

- Dispositivo con Android 7.0 o superior.
- Conexión a internet (opcional, app *offline-first*).
- Espacio de almacenamiento disponible.

Anexo III: Cronograma del Proyecto

Figura 4: Cronograma general del desarrollo.

Como se observa en el cronograma, el esfuerzo principal de ingeniería (Core y UI) se concentra entre noviembre y enero, reservando febrero para la lógica compleja de IA y marzo exclusivamente para la estabilización (QA y Testing), asegurando así llegar a la defensa con un producto robusto y libre de errores críticos.